Knowledge-Based Systems 210 (2020) 106420

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys

Incremental learning based multi-domain adaptation for object detection

Xing Wei, Shaofan Liu *, Yaoci Xiang, Zhangling Duan, Chong Zhao, Yang Lu

Hefei University of Technology, China

ARTICLE INFO

Article history:
Received 12 March 2020
Received in revised form 10 August 2020
Accepted 1 September 2020
Available online 18 September 2020

Keywords:
Domain adaptation
Object detection
Incremental learning
Image processing

ABSTRACT

Cross-domain object detection uses knowledge from source domain tasks to enhance the object detection in target domain. It can reduce the workload of data annotations in the new domain and significantly improve the adaptation ability of the network. We consider a more realistic transfer scenario, that is, our target domain samples cannot be obtained at the same time so that learners cannot forget the old domain knowledge while learning the new domain knowledge, which is consistent with the basic assumption of incremental learning. However, the existing methods do not consider the sequential learning process of the multiple target domains which may cause performance degradation. To address this issue, we propose a novel multi-domain adaptation method for object detection based on incremental learning. Specifically, the incremental learning network saves the knowledge of multiple domains and makes the model to fuse the knowledge of different domains during the training effectively. To make it work better, the progressive training strategy is proposed to make the model gradually adapt to multiple domains. Moreover, we use a multi-level feature alignment module to ensure that domain alignment is realized on features at various levels. We perform experiments on two sets of 6 datasets, which demonstrate that our model can effectively solve the problem of domain knowledge forgetting in multi-target domain adaptation and significantly improve the detection accuracy in each domain.

1. Introduction

In recent years, with the development of deep neural networks, the detection accuracy and speed of object detection networks have been significantly improved. However, these successes heavily rely on massive amounts of labeled data. As large-scale data annotation is usually required vast manual labeling, it is a feasible method to train with labeled datasets from a different but related source domain. Unfortunately, the discrepancy between the training datasets and the test datasets is inevitable, which makes the model trained in the source domain occur substantial performance degradation if it directly uses in the target domain. The state-of-the-art solution is unsupervised domain adaptation (UDA). Models that are trained on benchmark datasets can predict well in the new domain.

The main approach of unsupervised domain adaptation is to bridge the source and target domains by learning the public feature subspace. As the research of Yosinski et al. [1] shows deep neural networks can mine more domain-invariant features to improve the network's adaptive ability, a large number of methods for domain adaptation based on deep neural networks have appeared [2–8] mainly solved the basic problem of image classification. In the domain adaptation for object detection, most methods based on adversarial training [9] to align edge feature distributions or instance-level distributions [10–12], and there are also some methods based on distribution learning [13] which aligns the image feature distribution by self-training.

Although the above domain adaptation methods for object detection effectively improves the detection performance on the target domain, it only considers the problem of alignment between a single source domain and a single target domain. The problem we considered belongs to multi-target domain adaptation which is a more practical scenario. It learns the public feature distribution that is not related to all domains and transfers knowledge from the source domain to multiple target domains so that the network can detect objects in multiple target domains at the same time. For example, we hope that an autonomous vehicle can run on various urban roads [14], and a robot can face several weather conditions [15]. The existing methods can only solve the multi-target domain adaptation problem by fine-tuning or merging multiple target domains into one target domain for joint training which obviously has disadvantages. For fine-tuning method, it is difficult for a fine-tuning method to keep existing

* Corresponding author.
E-mail addresses: weixing@hfut.edu.cn (X. Wei), frank-uzi@hotmail.com (S. Liu), xyc_1997@163.com (Y. Xiang), duanzl@mail.hfut.edu.cn (Z. Duan), zhaochong@mail.hfut.edu.cn (C. Zhao), luyang.hf@126.com (Y. Lu).

https://doi.org/10.1016/j.knosys.2020.106420
0950-7051/

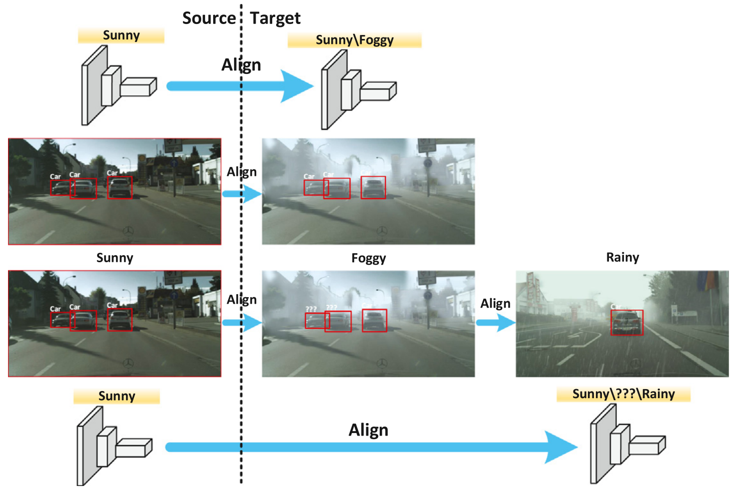


Fig. 1. Knowledge forgetting problem in multi-target domain adaptation. Firstly, the object detection model trained on sunny and foggy urban datasets can perform well on two domains (top). Secondly, when the model is retrained with rainy datasets, the detection performance in foggy scenes decreases (bottom).

domain knowledge when the feature discrepancy between target domains is large which may cause catastrophic forgetting [16]. As the training goes on, the network begins to forget the domain knowledge that has been learned, as shown in Fig. 1. Merging multiple target domains is not always effective, because in real scenes, many tasks cannot get all the training data at once, which makes the model must be able to use the constantly generated new data to learn new knowledge, and do not forget the important knowledge learned before. In summary, the problem we considered is closer to reality and has higher research value.

To address the issues above, we propose a novel multi-domain adaptation method for object detection based on incremental learning. We add the incremental learning network on the object detection network to save the knowledge of the previous domain and make the model to fuse the knowledge of different domains during the training effectively. Moreover, our model does not need previous domain samples when adapting to new domain. The incremental learning network consists of two parts. Firstly, it applies a feature transfer mechanism after the low-level convolutional layer and the middle-level convolutional layer and transfer the domain knowledge stored in the incremental learning network to the object detection network at different feature levels, so that the network can learn the previous domain knowledge. Secondly, we add a knowledge distillation function at the end of the bottleneck layer to ensure the consistency of high-level semantic information between the outputs of the two networks, so that the features input to the RPN network have domain-invariant characteristics on multiple target domains. Moreover, we propose a training method for incremental learning networks which can implement end-to-end training of the network on multiple domains. Through the constraints of the incremental learning network, the model can avoid serious domain knowledge forgetting problems when adapting with the domains of large discrepancy and promote to learn new domain distribution through existing domain knowledge when the domain discrepancy is small. Finally, we construct a multi-level feature alignment module which learns domain-invariant features by adding domain discriminators at different levels of the network.

Our main contributions are summarized as follows:

- We propose a solution for a multi-domain adaptive problem by designing an incremental learning network to avoid the problem of domain knowledge forgetting.
- An incremental learning training method is proposed to ensure end-to-end training of multi-domain adaptation.
- A multi-level feature alignment module is proposed to reduce feature discrepancy between domains by aligning features distribution at different levels.
- We perform domain adaptation experiments in various object detection scenarios. Our model effectively detects objects in multiple domains at the same time compared with baseline models.

The rest of the paper is organized as follows. Related works are briefly reviewed in Section 2. In Section 3, we elaborate on the Incremental learning based multi-domain adaptation for object detection. In Section 4, extensive experiments are performed on several domain adaptation tasks. Section 5 presents the conclusion and future work.

2. Related work

2.1. Domain adaptation

Since Yosinski et al. [1] revealed that features in deep neural networks are also transferable, most domain adaptive methods are mainly based on deep neural networks. They are divided into two categories: metric distance method and feature representation method.

The metric distance method is to minimize the discrepancy in feature distribution by constructing a suitable metric function. The most typical measurement method is MMD [17], which maps features to RKHS space for measurement. Long et al. [2] first extended MMD to a deep CNN model and achieved great success. They matched the shift in marginal distributions across domains by adding multiple adaptation layers. In contrast to this, Sun et al. extended the CORAL [3] metric to implement the domain adaptation of deep neural networks which learned a linear transformation that aligns the second-order statistics between domains.

Due to the success of GAN [9] networks, more and more domain adaptation methods adopt the idea of adversarial training and learn domain-invariant features by embedding domain discriminators into the network. Ganin et al. implemented adversarial training between the domain classifier and the classification network through the gradient reversal layer (GRL) [4] to make the distribution between the two domains more similar. In contrast to the above methods, Adversarial Discriminative Domain Adaptation (ADDA) [5] trained two feature extractors for the source and target domains, and produced embeddings to fool the discriminator. Moreover, generating target domain pseudo labels is also an effective method for domain adaptation. Saito et al. [6] introduced the labeling networks to assign pseudo labels to the target samples, which were subsequently used to train the final classifier for the target domain. TarGAN [7] is able to generate target data with given class labels, which can effectively boost the classifier's classification accuracy over the target domain. JMFL [8] used both metric distance method and feature representation method which simultaneously mitigates the data distribution and learns a feature representation via a common objective.

2.2. Target-agnostic DA

Besides the traditional problem of adaptation between a single source domain and a single target domain, there have been some works to study the Target-agnostic DA problem, that is, the target domain is comprised of multiple sub-targets implicitly blended with each other, so that learners could not identify which sub-target each unlabeled sample belongs to. This assumption is closer to the real migration scenario. Peng et al. [18] encodes the image to a Gaussian distribution through a variational autoencoder and performs Disentanglement on the category information and domain information. They also introduced Mutual Information to reduce the entanglement of two types of information. Chen et al. [19] proposed the Adversarial Meta-Adaptation Network (AMEAN), which uses an unsupervised meta-learner to accept target domain data and their feedback representations and learns to separate the mixed target domain into k meta-sub-targets. Liu et al. [20] proposed the open compound domain adaptation, aiming to solve the Target-agnostic DA problem and even open unknown domains. First, the image features are encoded into category features and domain features; Second, curriculum learning defines the learning process by the domain distance between target domain samples and source domain samples; Third, for the open unknown domains, they enhance the semantic representation of classification through a memory mechanism.

Unlike the above Target-agnostic DA, we studied the multi-target domain DA problem under the incremental learning setting. At this time, the labels of multiple target domains are known, but the model cannot obtain all target domain samples at the same time during training. It requires the model to learn the new domain knowledge and not forget the old domain knowledge.

2.3. Domain adaptation for detection

While domain adaptation has been extensively studied for cross-domain classification, cross-domain object detection is a more challenging and less studied task. The existing methods are mainly divided into two types. One is domain adaptation methods based on adversarial training; the other generates pseudo labels or intermediate domain images to train the network. Chen et al. [10] first adopted adversarial training for object detection domain adaptation. They argued that the gap between domains can be found both at the image-level (illumination, style) and the instance-level (object size and overall appearance), so they added image-level and instance-level domain classifiers based on Faster-RCNN [21] to adapt the detector to the target domain. On the basis of this, Saito et al. [12] argued that strong alignment of the features at global level might hurt the detection performance, so they proposed a strong-weak alignment network. In contrast, Zhu et al. [11] emphasized the importance of region-level alignment, and they proposed region mining and region-level alignment to align the source and target features correctly.

Besides the above adversarial training methods, Inoue et al. [22] tackled the cross-domain weakly supervised object detection task using a two-step progressive domain adaptation technique to fine-tune the detector trained on a source domain. They adopted CycleGAN [23] to convert the source domain images into the target domain images and fine-tune the network by transformed images. Yu al et. [13] continuously improved the accuracy of pseudo-labels by self-training and trained the network with modified pseudo-labels.

However, these methods consider the problem of domain adaptation only based on a single target domain. Our method aims to solve the problem of multi-domain adaptation and make the model perform well on multiple target domains. Based on Strong-weak DA, our model has been adjusted for multi-domain adaptation problems, and an incremental learning network has been added to preserve the knowledge of multiple domains with an incremental learning training strategy.

2.4. Incremental learning

Adding a new ability to a neural net often results in so-called "catastrophic forgetting" [16], and the best way to solve this problem is incremental learning. Polikar at el. [24] proposed a universally accepted definition of incremental learning. Incremental learning is a method of machine learning in which the learning model to adapt to new data without forgetting its existing knowledge, and it need not retrain the model. The work can be roughly divided into two categories depending on whether they require data from the old classes.

In methods without old data, EWC [25] proposed a technique to remember old tasks by constraining the important weights when optimizing a new task, but the old and new tasks may conflict on these important weights. Li et al. [26] earliest adopted knowledge distillation [27] to solve the catastrophic forgetting problem. The distillation loss reduces the discrepancy between the activations of past classes in the initial and the updated network. Similarly, we adopt distillation loss to align the semantic features of high layers in the network.

In methods that require old data, Castro et al. [28] kept classifiers for all incremental steps and used them as distillation. It introduced balanced fine-tuning and temporary distillation to alleviate the imbalance between the old and new classes. David Lopez-Paz et al. [29] proposed a continuous learning framework where the training samples for different tasks are used one by one during training. It constrains the cross-entropy loss on softmax outputs of old tasks when the new task comes.

In addition, the difference in class space between old and new tasks is also an important reason for catastrophic forgetting. For example, the long-tailed distribution will increase the difference in sample quantity between different tasks. Hou et al. [30] believe that this difference will cause three negative effects. First, when the number of samples in the new classes is large, the magnitudes of the weight vectors of new classes are remarkably higher than those of old classes; Second, when the number of samples in the new classes is small, the weight vector of the new class is close to the old class, resulting in ambiguity; Third, the sequential execution of new and old tasks leads to catastrophic forgetting. They solved the above problem through three steps of Cosine Normalization, Less-Forget Constraint and Inter-Class Separation and design a comprehensive loss function. It is also an effective method to reduce the influence of the long-tail distribution by designing a specific model structure and training strategy. Zhu et al. [31] proposed the Inflated Episodic Memory structure, which encodes different features in different regions and saves the encoded features and network prediction results in the form of key–value pairs in the bank to extract representative features. In order to analyze the influencing factors of the long-tail problem, Kang et al. [32] designed three different sampling strategies to learn feature representations, and studied different methods to obtain classifiers with balanced decision boundaries.

We specifically design an incremental learning network for the domain adaptation problem, which stores the weight of the old model and transfers it to the domain discriminator through the distillation function and feature transfer mechanism to enhance the model's discrimination of the old domain. In this paper, we only discuss the most common setting in domain adaptation, that is, the source and target domains have the same class space, ignoring the impact of class imbalance on domain adaptation.

3. Method

The structure of the incremental learning network is the same as the bottleneck layer. It is used to save the weights of the model trained on the previous domain. During training, the incremental learning network transfers the knowledge of the old domain to the multi-level feature alignment module by low-level and middle-level feature maps to avoid knowledge forgetting. Moreover, at the end of the network bottleneck layer, the distillation function is added to ensure the consistency of high-level semantic information. We construct a multi-level feature alignment module which adds domain classifier at different layers of the network to achieve the feature alignment of the image between the source and target domains. Our object detection network is based on the typical Faster-RCNN [21] network. The source and target images pass through the bottleneck layer, the RPN layer, and the classification regression network to obtain the detection result. In the rest of this section, we will introduce multi-level feature alignment modules (Section 3.2), incremental learning networks (Section 3.3), and training strategy (Section 3.4) in detail.

3.1. Formulation

The multi-domain adaptive problem assumes that there are two domains. The source domain with sufficient labeled data $D^s = \{x_i^s, y_i^s\}_{i=1}^{n_s}(x_i^s \in X^s, y_i^s \in Y^s)$, and multiple target domains with no labeled data. We assume that there are k target domains and each target domain has its own feature space $D_k^t = \{\{x_{i,k}^t\}_{i=1}^{n_t}(x_i^t \in X^t)\}_{k=1}^m$, where x_i indicates the input image, y_i indicates the image label, n^s and n^t indicate the number of images in the source and target domains, respectively. During training, the number of images in each target domain is the same. We randomly sample a fixed number of images in the target domain to ensure that this assumption holds. According to the basic assumption of unsupervised domain adaptation, the source and target domains follow different distributions, that is $P(X^s) \neq P(X_k^t)(k = 1 \dots m)$, and feature distributions are also different between each target domain $P(X_j^t) \neq P(X_k^t)(k \neq j)$. Our goal is to train a network that can make the feature distribution $f^s = F(x^s)$ of the source domain image closer to the feature distribution $f_k^t = F(x_k^t)(k = 1 \dots m)$ of the target domain image, that is, $P(f^s) \approx P(f_k^t)(k = 1 \dots m)$.

3.2. Multi-level feature alignment module

Chen et al. [10] analyzed the reason of feature discrepancy between different domains in object detection, and constructed image-level alignment and instance-level alignment in the network. Saito et al. [12] proposed Strong-weak DA which divides the domain adaptation into strong local alignment and weak global alignment. In order to further enhance the adaptation ability of the network, we add a middle-level feature map alignment module. Compared with Strong-weak DA, our model can capture more image features of different sizes to achieve more effective feature alignment. Therefore, we perform feature alignment on feature maps of three different sizes by domain classifiers, as shown in Fig. 2. We add a pixel-level domain classifier on the low-level and middle-level feature maps. It indicates which domain each feature point on the feature map belongs to and focuses on the local features of the image. For high-level feature maps, which mainly include the global features of the image, we classify it as a whole.

The structure of the feature alignment module is shown in Fig. 2. We add the GRL layer [4] to implement the adversarial training for feature alignment, that is, the domain classifier and the bottleneck layer network play against each other to generate domain-invariant features. In the pixel-level feature alignment module, the input features I_s and I_t are sent to the pixel-level domain classifier C_p after the GRL layer. C_p is a fully convolutional network with a convolution kernel size of 1. We train the classifier C_p by the mean-square-error loss. The loss function is as follows:

$$L_p^s = \frac{1}{n_s HW} \sum_{i=0}^{n_s} \sum_{h=0}^{H} \sum_{w=0}^{W} (C_p(I_{s_i})_{hw})^2 \tag{1}$$

$$L_p^t = \frac{1}{n_t HW} \sum_{i=0}^{n_t} \sum_{h=0}^{H} \sum_{w=0}^{W} ((1 - C_p(I_{t_i}))_{hw}{}^2) \tag{2}$$

where I_s and I_t represent the feature maps generated by the convolution block in the source and target domains, respectively. The width of the feature map is W and the height is H. We assume that the source image domain label is 0 and the target image domain label is 1.

The global feature alignment module is composed of a GRL layer and a global domain classifier C_g. C_g consists of a three-layer convolution block with a convolution kernel size of 3, an average pooling layer and a fully connected layer. The size of the feature map is halved after each convolution operation. Finally, the fully connected layer outputs the confidence that the feature map belongs to the domain. The loss function is as follows:

$$L_g^s = -\frac{1}{n_s} \sum_{i=0}^{n_s} (C_g(I_{s_i}))^\gamma \log(1 - C_g(I_{s_i})) \tag{3}$$

$$L_g^t = -\frac{1}{n_t} \sum_{i=0}^{n_t} (1 - C_g(I_{t_i}))^\gamma \log(C_g(I_{t_i})) \tag{4}$$

we adopt Focal Loss [12,33] to optimize the network to reduce the weight of easy-to-classify samples, making the model more focused on difficult-to-classify samples during training. In the experiment, we set the parameter γ is equal to 2 which is the same as the experimental setting in [12].

According to formula 5, we sum these loss functions to obtain the overall domain classification loss:

$$L_C = \frac{1}{3}(L_{p1}^s + L_{p1}^t + L_{p2}^s + L_{p2}^t + L_g^s + L_g^t) \tag{5}$$

where L_{p1}^s and L_{p1}^t represent the loss function of the low-level domain classifier, L_{p2}^s and L_{p2}^t represent the loss function of the middle-level domain classifier.

3.3. Incremental learning network

The traditional domain adaptation network represented by Strong-weak DA only considers the alignment problem between a single source domain and a single target domain. If they directly learn from multiple target domains, it will lead to "catastrophic forgetting". Therefore, on the basis of the traditional domain adaptation network, we added an incremental learning network

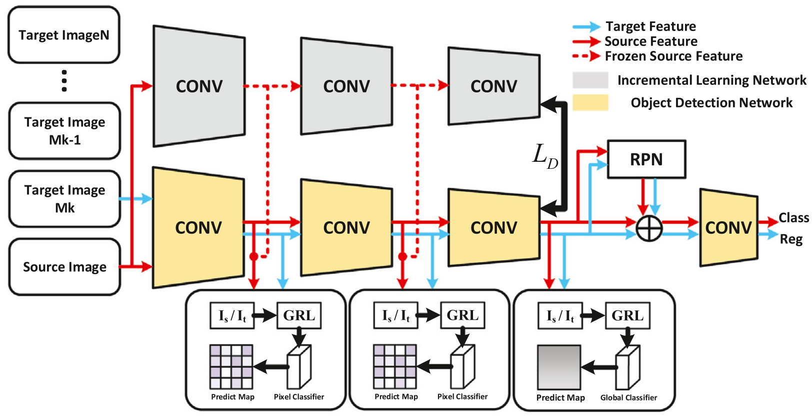


Fig. 2. Framework of Incremental learning-based multi-domain adaptation for object detection. The object detection network is composed of yellow convolution blocks outputs the classification and regression results. The multi-level feature alignment module aligns the feature distribution of the source and target domains at different levels. The incremental learning network is composed of gray convolution blocks to save the model weights after each adaptation alignment of the source and target domain. We freeze its weights to keep it not be impacted by network training. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

to preserve the knowledge of the old domains. The structure of the incremental learning network is the same as the bottleneck layer in object detection network, and it is composed of low-level, middle-level, and high-level convolution blocks. It implements incremental learning by saving old domain model weights and transfer them to the new model. It consists of two parts, the feature transfer mechanism and the distillation function. During training, we sum the features outputted by the incremental learning network with the object detection network features. Then, we send the features to the feature alignment module. We perform the feature transfer mechanism after the low-level and middle-level convolution blocks of the network, as shown by the dotted line in Fig. 2. The structure can be formulated as:

$$I_1 = F_1(x_0) + F_1'(x_0) \tag{6}$$

$$I_2 = F_2(F_1(x_0)) + F_2'(F_1'(x_0)) \tag{7}$$

where F_1 and F_1' represent the low-level convolution blocks in the object detection network and the incremental learning network, respectively. Similarly, F_2 and F_2' represent the middle-level convolution blocks in these two networks, and x_0 represents the source domain image. We only feed the source domain images to the incremental learning network, so that the source domain features include old domain knowledge. Then, we pass source domain feature map I_1 and I_2 to the domain classifier, so that the domain classifier can get the old domain knowledge. Moreover, the knowledge distillation function is added at the end of the bottleneck layer and the incremental learning network to ensure the consistency of the semantic information outputted by the high-level network. We choose L2-norm as loss function because the feature outputted by high-level convolution block is not a classification layer but instead just an internal stage.

$$L_D = \left\| F_3(x) - F_3'(x') \right\|^2 \tag{8}$$

where F_3 and F_3' represent the high-level convolution blocks of the detection network and the incremental learning network, respectively. x and x' represent the input features of the two networks. Finally, we sum the above losses to obtain the final loss function:

$$L = L_O + L_C + \lambda L_D \tag{9}$$

we represent the loss function of the object detection network with L_O which contains classification loss and regression loss. λ is a hyperparameter that is used to adjust the degree of the distillation function affects the loss function.

3.4. Training strategy

During training, we freeze the incremental learning network weights to keep the old domain knowledge. At each round of iteration, we input the source and target images to the model. When the source domain image is inputted, as shown by the red line in Fig. 2, the image passes through the incremental learning network and the object detection network at the same time. The Incremental learning network inputs source domain images and outputs features with old domain knowledge. Then, the network passes these features to different levels to the domain classifiers in the detection network. These features can train the domain classifiers effectively to distinguish whether the features belong to source domain or old target domain. When the target domain image is inputted, as shown by the blue line in Fig. 2, the image only passes through the object detection network. At this point, the object detection network and the domain classifiers play a game with each other to align the feature distribution between the "source domain" and the target domain, where the "source domain" includes the old domain knowledge.

Now, we will show specific steps for training the model on multiple domains. In the initial stage, we train the model with labeled samples D_s in the source domain S and unlabeled samples D_{T_0} in the first target domain T_0. The trained model is represented by M_0. Then, incremental learning strategy is performed, that is, the network will train with a new target domain step by step. As shown in Fig. 3, we assume that now is the k round and the model's weights W_{Mk-1} of $k-1$ round is saved in the incremental learning network. We train the model with the samples in the source domain S and the target domain T_k, and get the weights W_{Mk}. Then, the incremental learning network saves the weights W_{Mk} and freezes them. Similarly, in the $k+1$ round, the model trained with the samples in the source domain S and the target domain T_{k+1}, and get the weights W_{Mk+1}. Finally, after alternating training, the network will effectively learn multiple domain knowledge. The training strategy is shown in Algorithm 1.

4. Experiments

In this section, we demonstrate the effectiveness of our model on multiple domains in two types of datasets. We compare with the state-of-the-art unsupervised domain adaptation methods for object detection, including DA Faster-RCNN [10] and Strong-Weak DA [12]. The samples of the detection results of our domain adaptive model on the baseline datasets are shown in Fig. 8.

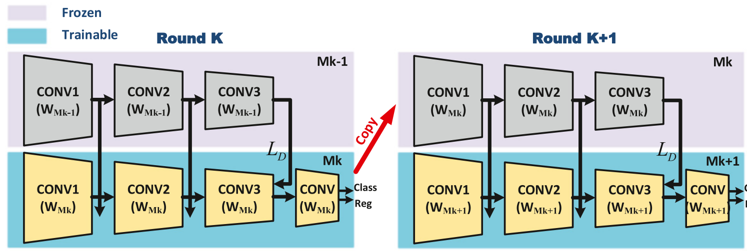


Fig. 3. Overview of the *k*th incremental step of our learning framework.

Fig. 4. Examples from six datasets.

Algorithm 1: Training strategy of multi-domain adaptation for object detection

---

Input: Source domain datasets X_s , target domain datasets X_{t_k}, $k \in \{0,1,2,\ldots,N\}$, domain adaptation network F , incremental learning network $F^{'}$, domain discriminator D .

Output: Parameter set θ_{opt}

if k = 0 **then**
  F=F_{init},$F^{'}$= 0
  Train F,D with mini-batch from training set X_s
  Train F,D with mini-batch from training set X_{t_0}
  Get trained model M_0 and save to $F^{'}$($F^{'}$= M_0.)
end if
for $k = 1$ to $Nstep1$ **do**
  Train F , D with mini-batch from training set X_s
  Train F.,D with mini-batch from training set X_{t_k}
  Get trained model M_k .and save to $F^{'}$($F^{'}$= M_k)
end for

4.1. Datasets

In order to verify the detection effect of the model on multiple target domains, we experiment on two types of datasets. The sample of each datasets is shown in Fig. 4.

Pascal VOC series: There are three different datasets Pascal VOC [34], Clipart [22] and Watercolor [22]. We use Pascal VOC as the source domain datasets and the other two as the target domain datasets. Pascal VOC contains total of 20 categories objects of different types in the real scene. We use Pascal VOC 2007 + 2012 as the training datasets. Clipart datasets is composed of a large number of clip arts in the network. There are a total of 1000 training set images and 1000 test set images, which contain objects from 20 classes, which are the same as the classes in Pascal VOC datasets. Watercolor dataset is composed of art painting images and contains total of 6 categories objects: bicycle, bird, cat, car, dog, and person. These categories are all included in the 20 categories of Pascal VOC datasets. The datasets contains a total of 17,814 images, we randomly select 2000 images from each as the training datasets and test datasets.

Cityscapes series: There are three different datasets Cityscapes [14], Foggy CitysScapes [35] and Rainy CitysScapes [36]. We use Cityscapes as the source domain datasets and the other two as the target domain datasets. CityScapes datasets contains 5000 images of driving scenes in an urban environment. This large datasets contains multiple stereo video sequences that were recorded of street scenes from 50 cities and taken during daytime and fine weather conditions. These 5000 images are divided into 2975 training datasets, 1525 test datasets, and 500 validation datasets, with a total of 8 categories. To obtain Foggy CityScapes datasets, digital image processing technology was used to generate images of foggy scenes that were based on CityScapes datasets. Foggy CityScapes inherits the complete labeling information of CitysScapes datasets. Rainy CitysScapes adopted the similar image processing technology to simulate the effects of rain and fog in images and generated a variety of different sizes of rain and fog effects for each image.

4.2. Experimental settings

1. We adopt ResNet101 [37] as the bottleneck layer network which has deeper layers than other networks and can extract sufficient features for domain adaptation training. We divide the ResNet network into three parts, which correspond to different levels of the bottleneck layer network. In the experiment, we pre-train the ResNet-101 models on ImageNet datasets [38] and fine-tune the model with labeled source domain samples and unlabeled target domain samples. For the experimental settings, we adopt the same settings as [10]. Before training, we limit the side length of the input image to between 600 and 1200 pixels, and we pre-process the image using a random flipping method. We use only one source domain image and one target domain image every iteration. During training, we utilize mini-batch SGD with momentum 0.9 and adopt the learning rate strategy to reduce the learning rate by a factor of 10 for every six epochs. In the test phase, we set the threshold of the intersection over union (IOU) to 0.5, namely, if the ratio of the area of intersection between the detection box and the annotation box exceeds 0.5, we regard the detection as a correct detection. We evaluate all categories in the datasets using average accuracy AP and mAP.
2. Because the existing baseline model does not consider the multi-domain adaptation problem, we perform two different training methods on the baseline model for accurate comparisons. One method is not to use incremental learning strategies, that is, the knowledge or weights of the old domains are not used when the model is adaptive to the new domain. Another method is to use an incremental learning strategy with fine-tuning (the results with the symbol **'FT'** in the table), we fine-tune the model with the weights of the old domains when training with the new domain. The training order of the model on the target domain does not affect the experimental conclusions. We perform the experiments in the order shown in the table. In addition, we also extract the same number of images from multiple target domains to construct the mixed target domain, and use Strong-weak DA, DA-Faster and our model (removing the incremental learning network) to adapt to this new domain (the results with the symbol **'M'** in the table). However, the mixed target domain is contrary to the order adaptation method proposed in this paper, that is, all target domain images can be obtained at the same time during model training, we only verify the effectiveness of the proposed incremental learning strategy through this result. All models adopt the same experimental settings. Finally, we evaluate these models on multiple target domains.
3. Compared with the general incremental learning method, our method is more suitable for the sequential adaptation of multiple target domains. To prove this point, we compare it with the incremental learning framework in [26]. Specifically, we follow the training method in [26] and embed the incremental learning framework [26] into the domain adaptive object detection model as shown in Fig. 5. Before training, the new task data pass through the original network (θ_s and θ_o) to obtains the output Y_o (the result of classification and regression), and then we randomly initialize the new task classification layer parameter θ_n. During training, we freeze parameters θ_s and θ_o and train θ_n to converge, and then jointly train θ_s, θ_o, and θ_n until convergence. We adopt the following loss function to update the model parameters:

$$\widehat{Y_o} = Model(X_n, \widehat{\theta_s}, \widehat{\theta_o}) \quad (10)$$

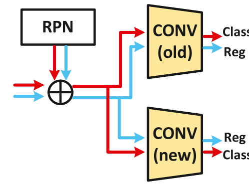


Fig. 5. Modified classification and regression networks to make it suitable for domain adaptive object detection problem.

$$\theta_s^*, \theta_o^*, \theta_n^* \leftarrow \operatorname{argmin}(\lambda L_{old}(Y_o, \widehat{Y_o}) + L + R(\widehat{\theta_s}, \widehat{\theta_o}, \widehat{\theta_n})) \quad (11)$$

$$L = L_o + L_c \quad (12)$$

The formula 10 indicates that the prediction result of the new task data obtained by the old model (the output of CONV(old) in Figure). $L_{old}(Y_o, \widehat{Y_o})$ in formula 11 is the distillation loss function, which is used to ensure the consistency of the model output on the old task. R is the regular term of stochastic gradient descent. L_o and L_c in formula 12 are the object detection loss and domain classification loss in the formula 9, respectively. We evaluate the model by the same experimental settings, and the **'Lwf'** symbol in Tables 2.2 and 3.2 indicates the evaluation result of the incremental learning model [26].

4.3. Results

In the first stage, the models are trained with the new domain, so we do not enable the incremental learning strategy. The detection results are shown in Tables 2.1 and 3.1, the symbol *Source Only* indicates the result of the baseline Faster-RCNN without any adaptive module. As we can see, all models achieve better detection result and our model is 2% mAP higher than the baseline models. Especially compared with Strong-weak DA, our model captures more differences between the image features of the source and target domains by further adding the middle-level feature alignment module, which is 2.1% mAP higher than the detection results of Strong-weak DA.

In the second stage, the models are trained with the other target domain and detect on the two target domain. The results are shown in Tables 2.2 and 3.2. We find that the baseline model without any incremental learning strategy has a significant decrease in detection accuracy in the old domain compared to the first stage. The accuracy dropped by up to 5% on the Clipart datasets and up to 9% on Foggy Cityscapes datasets. It indicates that the models that do not adopt any incremental learning strategy easily cause the problem of domain knowledge forgetting. When the model adopts incremental learning strategies, the detection results of Strong-Weak DA, DA-Faster, and Diversify in the old domain have all declined significantly (decrease by 8.6%, 7.2%, and 10.2%). It means that none of these traditional adaptation models can preserve the knowledge of the old domain while adapting to the new domain. Although the accuracy of our model has also decreased, the degree of decrease is less than the baseline model (4.9%), which means the proposed incremental learning network can effectively mitigate the degradation of model performance when the target domain has a large discrepancy. The most important thing is that while learning the knowledge of the old target domain, it does not affect the training with the new target domain. The model achieves the best detection results on both Watercolor and Rainy Cityscapes datasets, which are 52.8% mAP and 40.7% mAP, respectively. In Table 2.2, when the model finishes learning the first target domain (Clipart) and continues

Table 1
Detection results of models on the source datasets.

Pascal VOC datasets																				
Methods	aero	bicycle	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	hrs	bike	prsn	plnt	sheep	sofa	train	tv	mAP
Fast-RCNN	76.5	79.0	70.9	65.5	52.1	83.1	84.7	86.4	52.0	81.9	65.7	84.8	84.6	77.5	76.7	38.8	73.6	73.9	83.0	72.6	73.2
DA faster	77.3	78.1	74.3	63.7	59.2	83.7	86.3	86.9	53.9	82.4	64.2	83.3	84.8	78.8	78.1	46.8	75.0	67.7	82.9	75.4	74.1
Strong-weak DA	78.1	79.9	78.2	69.8	63.4	85.2	86.8	88.1	56.3	85.1	66.6	85.9	85.1	82.4	80.0	48.7	81.5	75.2	85.3	74.1	76.8
Proposed	78.8	81.7	77.9	68.2	65.6	86.6	86.4	88.9	57.0	82.5	67.6	85.9	85.9	82.0	79.4	49.7	79.7	73.1	84.4	78.1	77.0

Table 2.1
Detection results of domain adaptation from the Pascal VOC datasets to the Clipart datasets in the first stage.

Clipart datasets																				
Methods	aero	bicycle	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	hrs	bike	prsn	plnt	sheep	sofa	train	tv	mAP
Source only	35.6	52.5	24.3	23.0	20.0	43.9	32.8	10.7	30.6	11.7	13.8	6.0	36.8	45.9	48.7	41.9	16.5	7.3	22.9	32.0	27.8
DA faster	17.2	42.8	21.0	13.5	25.8	48.1	20.0	18.3	22.6	12.4	13.8	16.7	13.6	56.1	36.6	39.1	6.1	15.3	36.6	33.7	25.5
Diversify	23.7	49.4	28.5	29.0	34.6	52.4	28.0	6.9	40.4	38.4	16.7	27.1	45.3	80.3	63.8	35.5	16.5	16.8	36.9	43.5	35.7
Strong-weak DA	26.2	48.5	32.6	33.7	38.5	54.3	37.1	18.6	34.8	58.3	17.0	12.5	33.8	65.5	61.6	52.0	9.3	24.9	54.1	49.1	38.1
Proposed	36.9	49.5	39.8	21.0	39.7	63.7	34.8	29.0	39.4	55.6	12.2	28.2	24.7	82.1	56.7	42.4	12.1	26.3	54.8	54.7	40.2

Table 2.2
Detection results of domain adaptation from the Pascal VOC datasets to the Watercolor datasets in the second stage. The categories that do not exist in the Watercolor datasets are marked with the symbol '/'. The symbol 'FT' indicates training the model with fine-tuning strategy. The symbol 'M' indicates the results of the model training on the mixed target domain, otherwise it indicates training the model without any incremental learning strategy. The'Lwf' symbol indicates the evaluation result of the incremental learning model [26].

Clipart datasets																				
Methods	aero	bicycle	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	hrs	bike	prsn	plnt	sheep	sofa	train	tv	mAP
Source only	35.6	52.5	24.3	23.0	20.0	43.9	32.8	10.7	30.6	11.7	13.8	6.0	36.8	45.9	48.7	41.9	16.5	7.3	22.9	32.0	27.8
DA faster	11.8	46.7	17.0	12.9	23.6	41.9	19.3	13.1	23.4	11.7	4.5	22.1	15.0	44.0	33.0	38.2	2.5	16.5	26.3	40.0	23.2
Diversify (FT)	13.8	48.4	23.3	18.4	28.5	18.4	22.7	7.4	38.5	14.4	14.1	4.3	35.0	57.7	49.2	29.1	7.4	9.5	30.3	40.5	25.5
DA faster (FT)	20.8	40.9	18.5	13.0	25.3	36.7	19.6	10.1	25.2	16.4	9.7	16.4	12.0	47.3	42.5	38.5	3.4	15.2	31.4	34.1	23.9
DA-faster (M)	22.0	29.3	18.5	17.3	20.4	41.0	25.4	10.7	19.6	20.3	4.4	14.5	11.7	6.7	42.2	37.9	0.5	17.8	30.8	45.6	21.8
Strong-weak DA	35.3	56.0	26.3	17.7	32.8	48.2	31.7	12.9	36.3	26.2	13.7	18.0	23.4	70.2	47.9	41.6	6.2	26.8	51.4	50.9	33.7
Strong-weak DA (FT)	30.8	49.7	28.7	15.4	33.7	53.3	29.4	17.4	40.2	33.3	21.2	18.4	25.7	82.9	52.0	45.3	7.3	37.6	52.6	52.6	36.4
Strong-weak DA (M)	30.5	49.5	37.8	18.6	42.4	55.4	34.8	24.6	37.9	38.8	16.3	21.8	22.3	98.4	56.2	38.7	5.6	37.8	47.6	54.6	37.5
Lwf (IL method)	28.6	57.9	30.1	19.9	38.6	60.2	34.5	22.8	40.8	48.1	14.6	21.3	25.2	90.0	51.7	48.4	5.7	37.1	51.2	54.1	39.0
Proposed (M)	30.2	60.5	35.9	20.3	42.8	6832	36.9	24.5	41.0	52.3	12.2	25.7	25.6	89.6	57.4	50.6	7.8	38.6	58.0	54.7	41.6
Proposed	29.3	62.3	34.2	19.2	44.4	72.1	35.6	23.1	40.9	50.7	12.6	21.5	25.6	93.3	57.0	51.2	7.6	39.2	57.9	54.9	41.6
Watercolor datasets																				
Source only	/	68.8	46.8	/	/	/	37.2	32.7	/	/	/	21.3	/	/	60.7	/	/	/	/	/	44.6
DA faster	/	62.1	29.8	/	/	/	47.7	22.4	/	/	/	13.5	/	/	34.9	/	/	/	/	/	35.0
DA faster (FT)	/	82.2	34.5	/	/	/	46.5	20.8	/	/	/	14.7	/	/	50.0	/	/	/	/	/	41.5
DA-faster (M)	/	64.6	38.7	/	/	/	47.1	21.2	/	/	/	17.6	/	/	55.0	/	/	/	/	/	40.7
Diversify (FT)	/	65.3	42.1	/	/	/	49.8	36.2	/	/	/	28.7	/	/	61.6	/	/	/	/	/	47.3
Strong-weak DA	/	64.2	51.4	/	/	/	50.3	36.7	/	/	/	34.0	/	/	66.5	/	/	/	/	/	50.5
Strong-weak DA (FT)	/	81.3	51.2	/	/	/	46.6	38.8	/	/	/	33.7	/	/	63.7	/	/	/	/	/	52.5
Strong-weak DA (M)	/	59.4	52.9	/	/	/	50.4	39.7	/	/	/	34.1	/	/	66.6	/	/	/	/	/	52.7
Lwf (IL method)	/	81.2	52.0	/	/	/	48.7	36.2	/	/	/	32.0	/	/	64.3	/	/	/	/	/	52.4
Proposed (M)	/	81.6	53.1	/	/	/	50.2	35.8	/	/	/	32.6	/	/	64.8	/	/	/	/	/	53.0
Proposed	/	82.0	52.8	/	/	/	49.5	36.0	/	/	/	31.5	/	/	65.1	/	/	/	/	/	52.8

Table 3.1
Detection results of domain adaptation from the Cityscapes datasets to the Foggy Cityscapes datasets in the first stage.

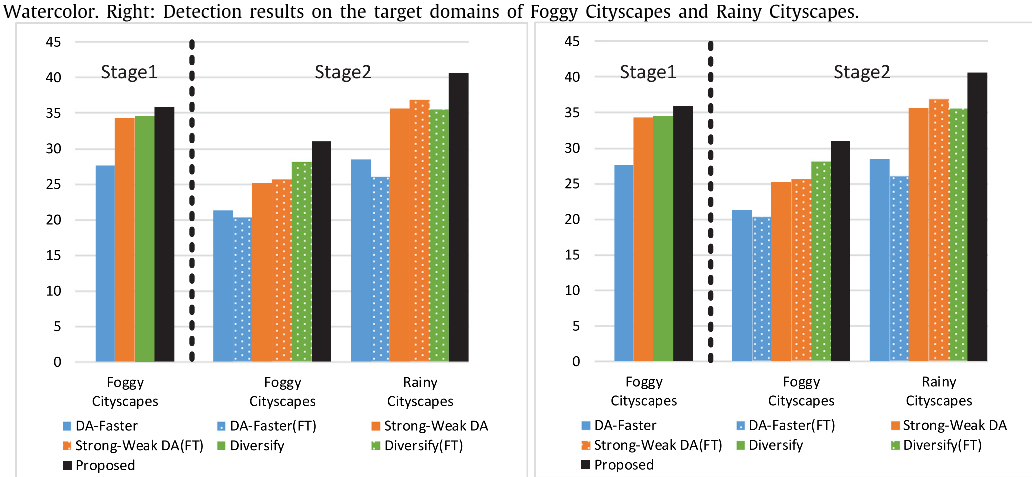
Foggy Cityscapes datasets								
Methods	bus	bicycle	car	bike	prsn	rider	train	truck	mAP
Source only	22.3	26.5	34.3	15.3	24.1	33.1	3.0	4.1	20.3
DA faster	25.0	31.0	40.5	22.1	35.3	20.2	20.0	27.1	27.6
Diversify	38.4	32.2	44.3	28.4	30.8	40.5	34.5	27.2	34.6
Strong-weak DA	36.2	35.3	43.5	30.0	29.9	42.3	32.6	42.5	34.3
Proposed	50.4	30.9	44.2	28.4	30.4	45.5	32.3	24.8	35.9

to learn the second target domain (Watercolor), the detection results of our model on Watercolor are not improved greatly (0.3%) compared with the baseline Strong-Weak DA (FT). This is because the discrepancy between the Watercolor dataset and the source domain PascalVOC dataset is small, both our model and Strong-Weak DA (FT) can reach a better level. However, the indicator that can highlight the superiority of our model is the detection accuracy in the old domain. The detection accuracy of the baseline model on the first target domain Clipart has significantly decreased. This shows that the traditional domain adaptation model will cause the problem of knowledge forgetting when sequentially learning multiple target domains. On the contrary, our model does not suffer from performance degradation on the old domain. Moreover, we find an interesting phenomenon: when the discrepancy between the target domains is small, such as the Clipart and Watercolor datasets, our model can not only ensure that the detection accuracy of the old domain does not decrease, but also improve the accuracy (increased from 40.2% to 41.6%). It indicates that the incremental learning network can fuse new and old domain knowledge to improve detection accuracy. We

Table 3.2
Detection results of domain adaptation from the Cityscapes datasets to the Rainy Cityscapes datasets in the second stage.

Methods	bus	bicycle	car	bike	prsn	rider	train	truck	mAP
Foggy Cityscapes									
Source only	22.3	26.5	34.3	15.3	24.1	33.1	3.0	4.1	20.3
DA faster	22.3	23.1	34.8	16.6	23.7	29.2	9.1	11.4	21.3
DA faster (FT)	15.3	24.9	34.0	16.5	23.6	32.4	9.7	7.0	20.4
DA-faster (M)	21.7	26.0	34.0	15.0	24.4	32.6	9.3	12.6	21.9
Diversify (FT)	36.5	32.1	38.2	17.5	26.3	39.8	19.2	15.4	28.1
Strong-weak DA	32.0	25.7	31.7	20.8	25.0	37.8	10.9	18.0	25.2
Strong-weak DA (FT)	29.0	27.9	35.8	15.9	24.3	36.8	15.8	19.9	25.7
Strong-weak DA (M)	38.6	31.9	42.3	27.3	28.2	33.6	24.8	22.2	31.1
Lwf (IL method)	39.8	30.1	37.6	25.6	27.3	40.9	17.6	23.0	30.2
Proposed (M)	40.5	31.1	38.9	26.1	27.7	41.4	18.9	23.9	31.1
Proposed	40.8	30.5	39.1	25.7	28.6	41.1	19.0	23.4	31.0
Rainy Cityscapes									
Source only	0.0	0.0	41.6	0.0	0.0	0.0	0.0	0.0	5.2
DA faster	31.3	26.4	42.6	28.2	24.5	42.9	18.9	13.0	28.5
DA faster (FT)	26.1	28.7	33.5	23.9	25.1	42.5	19.8	9.6	26.1
DA-faster (M)	30.9	25.8	39.7	30.4	24.8	43.8	23.3	9.2	28.5
Diversify (FT)	38.6	32.4	40.5	39.9	31.2	45.8	23.7	28.1	35.0
Strong-weak DA	40.5	33.9	43.0	40.5	29.3	46.4	24.5	27.5	35.7
Strong-weak DA (FT)	41.4	39.0	44.7	33.3	29.3	49.5	29.7	28.1	36.9
Strong-weak DA (M)	47.5	37.8	45.0	33.7	33.7	49.9	29.5	32.3	38.7
Lwf (IL method)	43.0	36.7	45.2	42.1	31.6	51.0	36.9	36.0	40.3
Proposed (M)	43.5	37.6	44.9	43.7	32.0	51.8	36.1	35.6	40.7
Proposed	42.3	38.3	45.0	43.3	32.2	52.2	36.4	35.8	40.7

Table 4
The histogram of detection results in multiple domains. Left: Detection results on the target domains of Clipart and Watercolor. Right: Detection results on the target domains of Foggy Cityscapes and Rainy Cityscapes.



further observe the impact of incremental learning networks on the detection accuracy of each category. In the second stage, although the average detection accuracy has improved, it does not mean that the accuracy of each category has also improved, and the accuracy of some categories has also significantly decreased (such as the car and sheep). This implies that the impact on each category of the old target domain is different when fusing the knowledge of the new target domain.

As shown in Tables 2.2 and 3.2, for the model trained on the mixed target domain, although the detection results of the baseline model trained on the mixed target domain are greatly improved compared with the traditional adaptation method and fine-tuning learning strategy, the detection results of our method are still leading in multiple target domains, and can effectively save knowledge from multiple domains. Moreover, we find that the detection accuracy of the incremental learning baseline Lwf [26] on multiple datasets is lower than our model. We suspect that the possible reasons are as follows: the adversarial training mechanism is used between the domain classifier and the bottleneck layer (generator), so directly fixing the shared layer θ_s parameters will affect the model adaptation effect; our incremental learning network adopts the layered feature transfer mechanism and distillation mechanism, which can maintain more various features than a single distillation loss function at the network output layer; In our model, the old task features are not directly transferred to the bottleneck layer, but are added to the domain classifier features. Our model uses the domain classifier to adjust the weight of the bottleneck layer, which is better than the original method.

Finally, we also perform experiments on the source domain datasets to verify that the model adaptation process does not reduce the detection accuracy on the source domain, as shown in Table 1. Our model achieves an accuracy of 77.0% mAP on the source domain datasets (Pascal VOC).

In Table 4, we use a histogram to show the detection results of different models on multiple target domain datasets.

4.4. Analysis

Hyperparametric analysis: The analysis result of Hyperparametric λ in Eq. (9), as shown in Fig. 6. We study by seven sets

Table 5
Ablation analysis of model structure.

Source: Pascal VOC Target: Clipart → Watercolor			
Feature transfer	L2 loss	Clipart (mAP%)	Watercolor (mAP%)
✓	✓	41.6	52.8
	✓	39.0	51.4
✓		37.9	51.7
		37.7	50.3

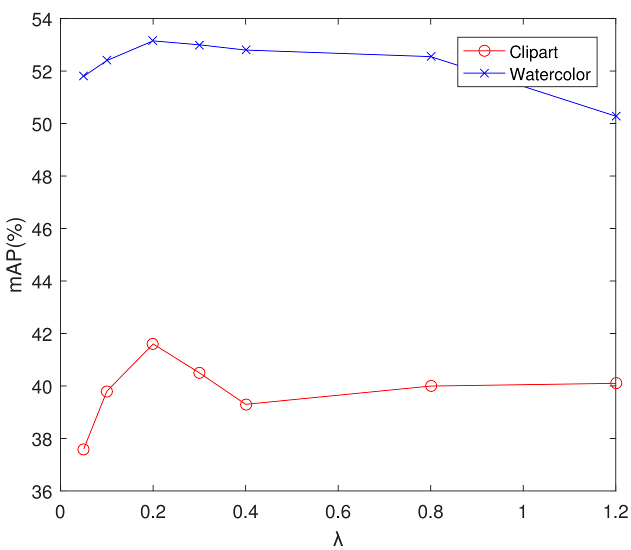


Fig. 6. Hyperparametric analysis.

of experiments with parameters λ ranging from 0 to 1.2, and show the detection results on two target domains, Clipart and Watercolor. When λ is 0.05, the detection accuracy on Clipart datasets is the lowest, because the L2 loss function accounts for a small proportion of total loss and difficult to effectively align the feature distribution of the incremental learning network and the detection network. Moreover, when λ is 1.2, that is, large proportion of L2 loss function, the detection accuracy on Watercolor datasets decreases sharply and it is easy to cause unstable training. We considered these two factors and choose 0.1 to 0.3 as the best value range of λ. At this point, the model training is stable and the model can obtain good detection results on both datasets. We set λ as 0.2 in the experiments.

Ablation study: In this part, we study the contribution of different components of the model to verify the impact of the incremental learning network on the experimental results. We show the detection results on two target domains, Clipart and Watercolor. As shown in Table 5, we can observe that each component is reasonably designed. The performance degrades when any component is removed. The model achieves the highest detection accuracy on two target domains when we add the feature transfer mechanism and L2 distillation loss at the same time. In contrast, the detection performance of the model decreases significantly when these two mechanisms are removed. We find that the incremental learning network can not only improve the detection accuracy in the old domain but also in the new domain. This is because, when the domain discrepancy is small, the knowledge learned from the old domain promotes learning in the new domain. Moreover, we also find that L2 distillation loss contributes the most to the performance improvement, and the feature transfer mechanism cannot significantly improve the model performance without distillation loss. When they work together, the model can reach the best performance.

Feature visualization: To demonstrate the feature transferability, we reduce the high-dimensional features output by the backbone network to low-dimensional planes by t-SNE [39], as shown in Fig. 7. The first row indicates that the 2-dimensionality feature distribution map when we input the new domain image (Rainy Cityscapes). All three models' feature distribution maps show that the feature distributions in the source and target domains are similar and the features are hard to separate, which indicates that the model has effectively learned domain-invariant features. The second row indicates that the 2-dimensionality feature distribution map when we input the old domain image (Foggy Cityscapes). The features output by our model are still inseparable, but the feature distributions output by two baselines models show a significant discrepancy with clear separable boundaries. This further illustrates that the baseline models have the problem of domain knowledge forgetting in the old domain.

5. Conclusion and future works

In this paper, we have proposed a novel multi-domain adaptation method for object detection based on incremental learning which the model trained in the source domain can be effectively transferred to multiple unlabeled target domains. The existing methods mainly align with a single source domain and a single target domain and do not consider the adaptation strategy in multiple domains. Our method can effectively learn multiple domain knowledge by incremental learning network which is the core component of the model. It guides model training without forgetting old domain knowledge by the feature transfer mechanism, and ensures the consistency of high-level semantic information between the new and old domains by the distillation function. Moreover, we also propose a multi-domain adaptation training strategy that suitable for our model, so that the network can be trained end-to-end. Compared with the state-of-the-art domain adaptation models, our model can improve the detection accuracy on multiple target domains. In the future work, we will continue to study the problem of domain adaptation based on target detection, and explore non-adversarial training domain adaptation methods for object detection, such as learning the feature distribution of target domain by pseudo labels and using self-training to optimize the pseudo labels.

CRediT authorship contribution statement

Xing Wei: Conceptualization, Methodology, Software. **Shaofan Liu:** Conceptualization, Methodology. **Yaoci Xiang:** Validation, Investigation, Visualization. **Zhangling Duan:** Writing - original draft. **Chong Zhao:** Writing - review & editing. **Yang Lu:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by Anhui Provincial Key R&D Program (201904d08020008) and National Key Technologies R&D Program of China (2018YFC0604404).

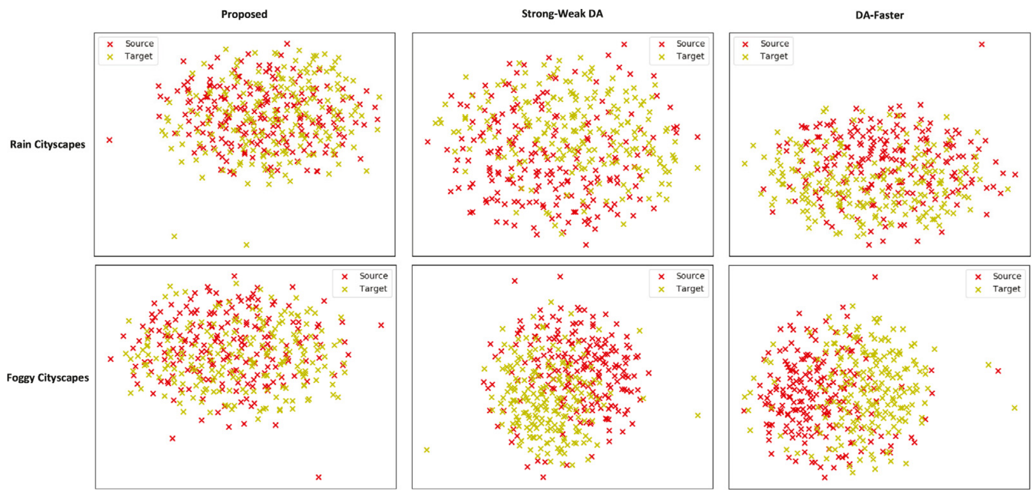


Fig. 7. The t-SNE visualization of feature representations. We randomly sample 200 images form Rainy Cityscapes and Foggy Cityscapes datasets for analysis.

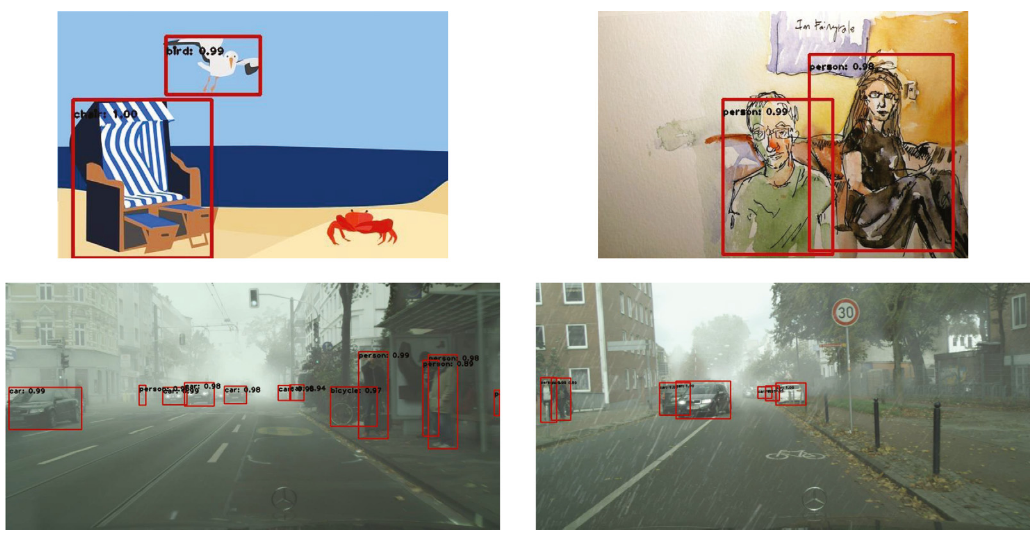


Fig. 8. Detection results of our model on different datasets. From top to bottom and left to right are Clipart, Watercolor, Foggy Cityscapes, and Rain Cityscapes datasets.

References

[1] J. Yosinski, J. Clune, Y. Bengio, H. Lipson, How transferable are features in deep neural networks? in: Proceedings of the Advances in neural information processing systems, 2014, pp. 33203328.

[2] M. Long, Y. Cao, J. Wang, M. Jordan, Learning transferable features with deep adaptation networks, in: Proceedings of the International Conference on Machine Learning, 2015, pp. 97105.

[3] B. Sun, J. Feng, K. Saenko, Return of frustratingly easy domain adaptation., in: Proceedings of the AAAI, 6, 2016, pp. 8.

[4] Y. Ganin, V. Lempitsky, Unsupervised domain adaptation by backpropagation, in: Proceedings of the International Conference on Machine Learning, 2015, pp. 11801189. Joint metric and feature representation learning for unsupervised domain adaptation.

[5] E. Tzeng, J. Hoffman, K. Saenko, T. Darrell, Adversarial discriminative domain adaptation, in: IEEE Conference on Computer Vision and Pattern Recognition.

[6] K. Saito, Y. Ushiku, T. Harada, Asymmetric tri-training for unsupervised domain adaptation[C], in: Proceedings of the 34th International Conference on Machine Learning-Volume 70, JMLR. org, 2017. pp. 2988–2997.

[7] F. Lv, J. Zhu, G. Yang, et al., TarGAN: Generating target data with class labels for unsupervised domain adaptation[j], Knowl.-Based Syst. 172 (2019) 123–129.

[8] Y. Xie, Z. Du, J. Li, et al., Joint metric and feature representation learning for unsupervised domain adaptation[J], Knowl.-Based Syst. (2019) 105222.

[9] I.J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio, Generative adversarial nets, in: NIPS, 2014.

[10] Y. Chen, W. Li, C. Sakaridis, et al., Domain adaptive faster r-cnn for object detection in the wild[C] in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2018, pp. 3339-3348.

[11] X. Zhu, J. Pang, C. Yang, et al., Adapting Object Detectors via Selec- tive Cross-Domain Alignment[C], in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2019, pp. 687-696.

[12] K. Saito, Y. Ushiku, T. Harada, et al., Strong-Weak Distribution Alignment for Adaptive Object Detection[C], in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2019 pp. 6956-6965.

[13] F. Yu, D. Wang, Y. Chen, et al., Unsupervised domain adaptation for object detection via cross-domain semi-supervised learning[j], 2019, arXiv preprint arXiv:1911.07158.

[14] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, B. Schiele, The cityscapes dataset for semantic urban scene understanding, in: CVPR, 2016.

[15] Y. Zhang, H. Chen, Y. He, M. Ye, X. Cai, D. Zhang, Road segmentation for all-day outdoor robot navigation, Neurocomputing 314 (2018) 316–325.
[16] Robert M. French, Catastrophic forgetting in connectionist networks, Trends in cognitive sciences 3 (4) (1999) 128–135.
[17] K.M. Borgwardt, A. Gretton, M.J. Rasch, H.-P. Kriegel, B. Schölkopf, A.J. Smola, Integrating structured biological data by kernel maximum mean discrepancy, Bioinformatics 22 (14) (2006) e49–e57.
[18] X. Peng, Z. Huang, X. Sun, et al., Domain agnostic learning with disentangled representations[j], 2019, arXiv preprint arXiv:1904.12347.
[19] Z. Chen, J. Zhuang, X. Liang, et al., Blending-target domain adaptation by adversarial meta-adaptation networks[C], in: Proceedings of the IEEE Con- ference on Computer Vision and Pattern Recognition, 2019, pp. 2248-2257.
[20] Z. Liu, Z. Miao, X. Pan, et al., Open Compound Domain Adapta- tion[C], in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 12406-12415.
[21] Shaoqing Ren, Kaiming He, Ross Girshick, Jian Sun, Faster r-cnn: to- wards real-time object detection with region proposal networks, in: NeurIPS, 2015, p. 9199.
[22] Naoto. Inoue, Ryosuke. Furuta, Toshihiko. Yamasaki, Kiyoharu Aizawa, Cross-domain weakly-supervised object detection through progressive domain adaptation, in : Proceedings of CVPR, pages 50015009, 2018. 1, 3.
[23] J.Y. Zhu, T. Park, P. Isola, et al., Unpaired image-to-image translation using cycle-consistent adversarial networks[C], in: Proceedings of the IEEE international conference on computer vision, 2017, pp. 2223-2232.
[24] R. Polikar, L. Upda, S.S. Upda, et al., Learn++: An incremental learning algorithm for supervised neural networks[J], IEEE Trans. Syst. Man Cybern., part C (applications and reviews) 31 (4) (2001) 497–508.
[25] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka GrabskaBarwinska, et al., Proc. Natl. Acad. Sci. 114 (13) (2017) 3521 3526.
[26] Zhizhong. Li, Derek Hoiem, Learning without forgetting, in: European Conference on Computer Vision, Springer, 2016, 614629.
[27] Geoffrey Hinton, Oriol Vinyals, Jeffrey Dean, Distilling the knowledge in a neural network, in: In NIPS Deep Learning and Representation Learning Workshop, 2015.
[28] Francisco M. Castro, Manuel J. Marin-Jimenez, Nicolas Guil, Cordelia Schmid, Karteek Alahari, End-to-end incremental learning, in: The European Conference on Computer Vision (ECCV), 2018.
[29] David Lopez-Paz, et al., Gradient episodic memory for continual learning, in: Advances in Neural Information Processing Systems, 2017, p. 64706479.
[30] S. Hou, X. Pan, C.C. Loy, et al., Learning a unified classifier incrementally via rebalancing[C], in: 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), IEEE, 2019.
[31] L. Zhu, Y. Yang, Inflated Episodic Memory With Region Self-Attention for Long-Tailed Visual Recognition[C], in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 4344-4353.
[32] B. Kang, S. Xie, M. Rohrbach, et al., Decoupling representation and classifier for long-tailed recognition[j], 2019, arXiv preprint arXiv:1910.09217.
[33] T.Y. Lin, P. Goyal, R. Girshick, et al., Focal loss for dense object detection[C], in: Proceedings of the IEEE international conference on computer vision, 2017, pp. 2980-2988.
[34] M. Everingham, L. Van Gool, C.K. Williams, J. Winn, A. Zisserman, The pascal visual object classes (VOC) challenge, IJCV 88 (2) (2010) 303338.
[35] C. Sakaridis, D. Dai, L. Van Gool, Semantic foggy scene understanding with synthetic data, IJCV (2018) 5, 6.
[36] X. Hu, C.W. Fu, L. Zhu, et al., Depth-attentional features for single-image rain removal[C], in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2019, pp. 8022-8031.
[37] K. He, X. Zhang, S. Ren, et al., Deep residual learning for image recognition[C], in: Proceedings of the IEEE conference on computer vision and pat- tern recognition, 2016, pp. 770-778.
[38] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M.S. Bernstein, et al., Imagenet large scale visual recognition challenge, Int. J. Comput. Vis. 115 (3) (2015) 211–252.
[39] L. van der Maaten, G. Hinton, Visualizing Data using T-SNE.JMLR., 2008.